

PROJECT AND TEAM INFORMATION

Project Title

Smart University Timetable Management System

Team Information

<i>Team Name:</i> <i>Team #</i>	<i>Catalyst</i>
Team member 1 (Team Lead)	<i>Tripathi, Shreya-</i> <i>2510011705</i> <i>shreyatripathi1692@gmail.com</i> 
Team member 2	<i>Kanupriya-2510016460</i> <i>kanupriya1220@gmail.com</i> 
Team member 3	<i>Nautiyal, Medha-</i> <i>2510012378</i> <i>medhanautiyal06@gmail.com</i> 
Team member 4	<i>Abhimanyue Kush, Avika-</i> <i>2510012834</i> <i>kush.avikaa@gmail.com</i> 

PROPOSAL DESCRIPTION

Motivation

Timetable generation in schools and colleges currently happens mostly through manual processes done by the staff. As the number of sections increases, the task becomes very difficult to do. In our college, since there are numerous sections, subjects and teachers, creation of timetable through manual processes becomes very difficult and cumbersome.

Firstly, the major problem is that of conflicts. At times, due to some error, two lectures may be assigned to a particular teacher at the same time or a particular room is allotted to two separate lectures at the same time. Also, it becomes difficult for the staff to decide as to which teacher is free at a certain time and which one is not free.

Another problem is that if even the time of one class is changed, the whole timetable needs to be again reviewed manually to prevent conflicts. Time gets wasted and there could be errors as well.

Due to all these problems, the idea came up of designing an automated system called the 'Smart University Timetable Management System' which would automatically generate a timetable without manual intervention. The input to the system would be the details of teacher availability, subjects and sections and then a conflict free timetable would be generated by the system.

One more feature that we decided to include in our system is the inclusion of teacher IDs in the system. This would help in notifying the teachers regarding their schedule such as which periods are free for them and whether there is any change in their classes etc. This would bring organization in the process and eliminate the communication gap between the office staff and the teachers.

Currently, we are designing the initial prototype of the system using data structures in C++.

State of the Art / Current solution

In many colleges and schools, timetables are made manually by means of Excel or even by means of paper registers. In this case, all depends on the individual who makes a schedule, so mistakes are typical. It may involve assigning the same teacher to two classes at the same time or assigning the same room to two groups at the same time. In the case of any changes, it is necessary to go through the entire timetable manually again-a rather laborious process.

There are also colleges using software for schedule, but most of them are templates and all data should be entered manually. Such softwares do not solve the problem of constraints like teacher clashes, room clashes, subject load etc., so the manual checking of all these things is necessary.

Big institutes and universities use an ERP system including a timetable generator as well as attendance, fees etc. functions. Such a system works perfectly but costs much money, and not all colleges can afford purchasing such a system.

That is why it is obvious that there is a necessity of making a system which is simple and cheap enough to develop, and which will solve the problems of clashes and scheduling.

Project Goals and Milestones

The main objective of the project is to develop a Timetable Generation System which can automatically produce a timetable for a college whenever there are a large number of departments (sections), a wide range of subjects, and a large number of teachers. Instead of having to prepare the timetable by hand, the system will accept as input some basic information, including the teachers' availability, the lists of subjects, and the details of the sections, and will then use data structures in C++ to generate a valid timetable; it will also assign each teacher an ID for the purpose of tracking their timetable.

It can be employed in the later stages to remind teachers of their free periods or of any changes concerning their classes, thus eliminating the need to search for registers from time to time.

The first thing to do would be to gather information concerning the number of sections in the college, the subjects required by each section, and which teacher is in charge of teaching each particular subject. The system then had to be designed so that the data could be stored properly, using various data structures such as arrays, structures, and linked lists. Once the design stage was completed, the logic had to be written in order to prevent timetable conflicts by checking for double bookings of a teacher or of a particular section in the college. The next step would be to carry out the actual process of preparing the timetable, after which it would be shown as the output.

Consequently, each teacher will be assigned a unique ID, after which the teacher's ID will be linked to the timetable for that teacher in the next stage.

Before preparing the report and the presentation, the Timetable Generation System is to be tested with a sample set of data, the purpose of which is to illustrate the mechanism of the system, its design, and the results it has obtained.

Project Approach

- The project will be developed as a modular Smart University Timetable Management and Real- Time Notification System using C/C++, Object-Oriented Programming, Data Structures and The system will adopt a structured approach involving algorithms, STL, and file handling/database management. approach in which individual modules are developed independently and integrated through a central main driver/controller module.*
- The system will begin with user authentication and role-based access for students, faculty mem- And also those of the administrators. The records relating to users, courses, classrooms, de- partments, and sections, and timetables will be managed using classes, objects, arrays, vectors, linked lists, maps, and They allow for efficient insertion, deletion, updating, and searching as well as sorting.*
- The timetable management module will allow authorized administrators to create and modify by assigning the subjects, the staff, the classrooms, the sections, the days, and the time slots. A dedicated cated conflict detection module will identify faculty, classroom, section, duplicate, and time-slot conflicts before publishing changes.*
- Various data structures such as stacks,queues,priority queues,vectors,lists,maps and graphs will be incorporated for timetable management,notification handling,modification history,resource The allocation and efficient searching can be achieved. Trees and graphs may also be used to represent a university. The hierarchical relationships existing between courses, faculty members, classrooms, and sections.*
- C++ OOP concepts including encapsulation, abstraction, inheritance, polymorphism, construc- tors, destructors, function overloading, operator overloading, virtual functions and friend func- tions will be implemented through classes such as "Person", "Student", "Faculty", "Course", "Class- room", "TimeSlot" and "Timetable".*

Finally, file handling/database storage, exception handling, dynamic memory allocation,testing and report generation will be implemented to create a reliable, scalable, and user-friendly aca- demic scheduling system.

System Architecture (High Level Diagram)



Project Outcome / Deliverables

Timetabling is still done manually in many colleges through Excel or some software. This proposal suggests a Smart University Timetable Management and Real-Time Notification System which would help to manage: -students and teachers, courses and sections, classrooms and resources - timetables, timetable changes and academic notifications

Practical DSA topics: - arrays, structures, linked lists, stacks, queues, priority queues - maps, sets, trees, graphs - searching, sorting, timetable conflicts detection algorithms

C++ OOP topics: - classes/objects, encapsulation, abstraction - inheritance, polymorphism, constructors - function overloading, STL containers

A C/C++ web application with: - user authentication and role-based authorization for students, teachers, administrators - students and teachers management; courses, sections, classrooms, resources management - creation/modification/maintenance of timetables; automatic conflicts detection - synchronization of timetables in real-time and notifications about updates - history/versions of timetables management; files and databases handling - academic reports on timetables, resources and schedules

Possible advanced functions: - automatic timetables creation and resources allocation - dynamic memory management and exceptions handling - searching and sorting efficiently

Objectives: - reduce timetable misunderstanding and need for communication - prevent scheduling conflicts and errors - provide latest timetables to users - show practical usage of C/C++, OOP, DSA, STL, file handling, databases, real-time design

Result: a working, scalable, modular, user-friendly university timetabling system that solves actual problems of timetabling.

Background: at the moment there are several problems in timetabling: a lot of human mistakes (like booking the same teacher or classroom at the same time), need to recheck timetables after any change. At the moment existing software for solving this problem consists either of static templates or complex ERP systems. There is a necessity for cheap and simple system to solve timetable conflicts with basic programming and data structures.

Assumptions

- *User Authenticity Role Compliance:* The user supplied the login details during the sign-in process.
- *Faculty members, students and administrators all act within the permissions that have been given to them. For example only people with administrator rights can change the schedule and only faculty members can request to switch time slots.*
- *Data Consistency Standard Formats:* Identifiers such as Faculty ID, Course Code, Room Number and Section Name follow a format in every department.
- *Authorized people enter schedule changes, such as leaves or room repairs away.*
- *Resource Capacity Limits:* The maximum number of students in a class and the maximum number of classes a staff member can teach each day are set correctly in the system. It is not possible for a room or a person to be in two places at once.
- *Offline Execution and Local Storage:* The program is a tool written in C++.
- *Saving data in a permanent way uses simple local files, such, as.txt or.csv handled by std::fstream without needing cloud services or internet access.*

References

- Stroustrup, B. (2013). *The C++ Programming Language (4th ed.)*. Addison-Wesley Professional.
- Deitel, P., & Deitel, H. (2017). *C++ How to Program (10th ed.)*. Pearson.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms (3rd ed.)*. MIT Press. (For priority queues, hash mapping, and graph traversal logic).
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. (For Observer and Factory design patterns used in notifications and user roles).
- ISO/IEC. (2020). *ISO/IEC 14882:2020 Information technology — Programming languages — C++ (C++20 Standard)*. International Organization for Standardization.
- Cppreference. (2026). *C++ Standard Template Library (STL) Documentation*. Retrieved from <https://en.cppreference.com/>
- Burke, E. K., & Petrovic, S. (2002). *An overview of automated metalearning approaches for university timetabling problems*. *Annals of Operations Research*, 115(1), 123–146.
- *College Timetable Using Time Scheduling Algorithm*. IJERT. Retrieved from: <https://share.google/1MaBrNIjNEmGmpoD>